Implementation Document

for

MESS AUTOMATION SYSTEM

Version 1.0

Prepared by

Group 10: **Group Name: Byte Busters**

B Mahath	240252	bmahath24@iitk.ac.in
Priyanshi Meena	240804	priyanshim24@iitk.ac.in
Rishith Jalagam	240484	rishithjs24@iitk.ac.in
Eranki Venkata Jhanani	240385	vjhanani24@iitk.ac.in
Basavakiran Shivanand Deshmukh	240261	bshivanand24@iitk.ac.in
Inapakurti Rajesh	240463	rajeshr24@iitk.ac.in
Parth Parashar	240735	parthp24@iitk.ac.in
Shreyansh Dewangan	230987	shreyanshd23@iitk.ac.in
Rohan Kumar	230865	rohank23@iitk.ac.in
Siddhant Singh	231004	siddhant23@iitk.ac.in

Course: CS253

Mentor TA: Ujjwal Kajal

27 March 2026

Contents

CONTENTS..... 2
REVISIONS..... 3

1. IMPLEMENTATION DETAILS.....4

2. CODEBASE.....7

3. COMPLETENESS.....8

APPENDIX A - GROUP LOG.....10

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	B Mahath Priyanshi Meena Shreyansh Basavakiran Rishit Rajesh Parth Jhanani Rohan Siddhant	First Draft	27/03/26

1 Implementation Details

The Mess Management System is built on a specialized four-layer architecture, specifically designed to balance smooth web interactivity with the heavy lifting required for machine learning.

- **Presentation Layer:** This layer consists of the interface elements with which users directly engage. Built with React and TypeScript, it provides dynamic dashboards for students and managers, alongside real-time webcam integration for image capture during login and profile setup.
- **Application Layer:** This layer handles the core business logic. Built with Node.js and Express.js, it manages RESTful APIs, processes authentication requests, and orchestrates the flow of Base64 image strings between the frontend and the database.
- **Data Layer:** This layer is responsible for secure, persistent data storage. Utilizing PostgreSQL, it maintains relational data regarding student identities, billing, and stores user face snapshots directly as JSON/strings for quick retrieval and matching.

This structured approach allows for a clear separation of concerns, ensuring a highly responsive and scalable application.

1. Presentation Layer (Project Unit):

Technologies Used:

- **Programming Languages:**
  HTML & CSS: For structuring and styling the web pages.
  - **JavaScript / TypeScript integration:** The primary languages for interactive behaviour and strict type-safe state management in the browser**.**
- **Framework:**
  React.js: A JavaScript library for building modular user interfaces.
- **Libraries and Tools:**
  - **React-Webcam:** Captures real-time webcam feeds and generates base64 images directly within the browser.
  - **React Router:** Manages navigation between different views in the single-page application.
  - **Axios:** Handles asynchronous HTTP requests to interact with the Node.js backend.
  - **Vite:** A modern build tool and development server..

Justification of Choices:

- **React** was selected for its component-based architecture, which promotes reusability and easier maintenance across different dashboard views. Its Virtual DOM mechanism ensures efficient rendering, providing a smooth user experience even when handling live camera feeds.

- **React-Webcam** is crucial for this specific project because it natively hooks into the browser's media devices. It allows for seamless, real-time image capture required for the facial authentication flow (both during profile enrollment and login) without forcing the user to install third-party plugins.

- **Vite** was chosen over traditional bundlers like Webpack due to its significantly faster cold server start times and Hot Module Replacement (HMR). This drastically speeds up the frontend development and testing process.

- **React Router** simplifies navigation in a single-page application, while Axios provides a clean, promise-based approach to handle HTTP requests, making it easier to manage the transmission of large base64 image strings to the server.

2. Application Layer (Logic Unit)

Technologies Used:

- **Programming Language:**
  JavaScript (Node.js): Used for server-side development.
- **Framework:**
  Express: A minimalistic and flexible web framework for Node.js to create robust restful APIs.
- **Libraries and Tools:**
  - **CORS:** Middleware to enable cross-origin resource sharing between the frontend (Port 5173) and backend (Port 5000).
  - **pg (node-postgres):** A non-blocking PostgreSQL client for Node.js used to interact with the database.

Justification of Choices:

- **Node.js** was chosen because its non-blocking, event-driven architecture makes it exceptionally well-suited for handling heavy I/O operations, such as receiving large Base64 image strings from the frontend and processing them quickly.

- **Express** provides a minimalistic framework to build the routing logic. It easily handles the verification routes where incoming face snapshots are compared against the stored image strings from the database.

- **pg (node-postgres)** is utilized instead of a heavier ORM to provide direct, low-latency execution of SQL queries, while still maintaining a structured MVC folder architecture (src/models/). This ensures rapid synchronization and fast login times.

3. Data Layer (Storage Unit)

Technologies Used:

- **Database System:** PostgreSQL (configured with the facereco database).

Justification of Choices:

- **PostgreSQL** was selected for its strict ACID compliance, which is absolutely critical for a system handling financial transactions, mess dues, and rebate tracking. It ensures that complex relational data (linking unique face_id integers to student profiles) remains consistent across concurrent user sessions.
- **Direct Database Image Storage:** Instead of relying on a complex, external AI processing engine or managing local file system directories for images, user face snapshots are converted to JSON/Base64 strings and stored directly within the PostgreSQL students table. This greatly simplifies the architecture, improves deployment scalability, and ensures that user identity and biometric data are kept securely in a single, centralized location.

2 Codebase

We have made a repository on GitHub through which we store and collaborate on the source code of this project. You can access the repository here: https://github.com/CS253-Mess-Management/Final-codes

2.1 Repository Structure

The repository is organized into distinct functional directories, enforcing a strict separation between our independent frontend applicationsn and the backend API:

- **Mess_Management_Backend/**: Contains the core Node.js application (runs on Port 5000).
  - **src/models/**: Houses the data structure definitions mapping to PostgreSQL (e.g., Student.js, Vote.js, Transaction.js).
  - **src/controllers/ & src/routes/**: Contains the business logic, verification algorithms, and API endpoint definitions.
  - **src/config/ & src/middleware/**: Handles database connections and secure routing.
  - **src/utils :** In this there are additional functions or scripts which are used for mailing and cron jobs.

- **Frontend Applications (React + TypeScript):** Instead of a single monolithic frontend, the UI is modularized into three separate Vite/React projects utilizing TypeScript (.tsx) for strict type safety (running on Port 5173):
  - **Mess automation login page/**: The authentication entry point containing the react-webcam capture logic.
  - **Student Dashboard/**: Contains the specific views for student operations (voting, dues, rebates).
  - **Mess Manager dashbord/**: Contains the administrative views for managing the mess ecosystem.

2.2 Navigation Guide & Workflow

To trace the data flow of a user logging in, follow this sequence through the codebase:

Registration & Standard Login: Users initially register and log in using a standard ID and password via the Mess automation login page.

Photo Enrollment: Once authenticated, the user navigates to the Profile section within the Student Dashboard. By clicking "Add Photo", the react-webcam component triggers, capturing a snapshot of the user's face.

Data Storage: The frontend sends this snapshot to the Mess_Management_Backend/, where it is converted and stored in the PostgreSQL database as a JSON/Base64 string.

Subsequent Face Login: Upon returning to the Mess automation login page/, the user can opt for Face Login. The webcam captures a new snapshot and sends it to the backend.

Verification: The backend queries the PostgreSQL database (via src/models/), retrieves the stored image string, and processes the match. If successful, the user is instantly granted access and redirected to their dashboard.

3 Completeness

The Mess Automation System is a web-based application designed for the hostel community of the Indian Institute of Technology Kanpur (IITK) to digitalise mess operations. It provides a platform for managing student mess activities such as menu viewing, billing, extra food purchases, rebate requests, and feedback submission. Students can securely log in to view their mess details, participate in menu voting, track their expenses, and interact with mess management. Mess administrators can manage student registrations, menus, voting processes, billing records, and feedback through a dedicated dashboard. This application is focused on improving operational efficiency, reducing manual work, increasing transparency in billing, and making mess management more organised and user-friendly.

Features Implemented

I. User Authentication and Access Control:

A. Login

- Secure login using institute credentials
- Login through Face Recognition
- Role based access (Student / Manager)

B. Register

- Student registration using institute details
- Secure password creation
- Account activation and login

C. Forgot Password

- Password reset through registered email verification

II. Student Module:

A. Dashboard

- View today's mess menu (Breakfast/Lunch/Dinner)
- View Base Daily Mess Rate (BDMR)
- View monthly mess bill summary

B. Profile Management

- View personal details (Name, Roll No, Contact details)

- Update profile information like Contact details and email address

C. Extra Food Management

- Purchase extra food items
- Pre-book extra items
- Cart management(Add to cart and Update Cart)

D. View Dues

- View extra purchase costs
- Monthly Report Feature
- Transaction history tracking

E. Menu Voting System

- View active polls
- Vote for menu items
- View closed poll results
- View upcoming polls

F. Feedback System

- Submit feedback on food/service
- Star rating system
- View previous feedback
- Categorized feedback submission

G. Rebate Management

- Submit rebate requests
- View rebate history
- Track rebate history

III. Mess Manager Module:

A. Manager Dashboard

- Overview statistics
- Pending requests summary
- Quick management actions
- Recent activity tracking

B. ALL Students & New person requests

- Approve new registrations
- Reject requests
- View all students
- Search students

C. Menu Management

- Add or edit menus
- Manage weekly menu
- View Current Pre-Booking requests
- Manage active special items
- Tabs for Daily Menu, Pre-booking, Extra Items, and BDMR Settings

D. Poll Management

- Create polls
- Monitor voting results
- Start, End and Delete Polls

E. Rebate Requests

- Approve rebate requests
- Reject requests
- Track request status

F. Feedback Monitoring

- View student feedback
- Rating analytics
- Category filtering

G. Extra Buying History

- Extra purchase analysis
- Consumption insights
- Monthly reporting

Non-Functional Requirements Implemented:

A. Database Management:

- Centralized PostgreSQL database for storing:
  - Student records
  - Transactions
  - Voting data
  - Feedback
  - Billing information

B. Security Constraints:

- Role Based Access Control (RBAC)
- Secure authentication
- Password hashing
- Protected admin access

C. Communication Protocols:

- HTTP/HTTPS communication
- REST API architecture
- Client-server MVC architecture

D. Software Quality Attributes:

Reliability

- Structured layered architecture
- Error handling

Performance

- Fast API responses
- Efficient database queries

Usability

- Clean UI dashboards
- Easy navigation
- Role-specific interfaces

Scalability

- Modular architecture
- Separate frontend/backend layers

Future insights

- Sending an email to the Student at the end of the month consisting of the Total Mess bill of that particular month including the Extra Purchases.
- Recent Activity Tracking for Manager
- Manager can export Student List
- Total Dues Visible to a Student in View Dues Section (including pending)

Page

Appendix A - Group Log

- We have made a WhatsApp group for effective communication amongst ourselves.
- We have created a private repository on GitHub on which we would be regularly pushing our codes to maintain efficiency and collaboration.
- We broadly divided the team into parts, working on different aspects of frontend, backend, and database modelling so that all members understand the mechanism of software development and are able to implement it.

The following is the group log for the development of this Implementation Document

Date	Meet type	Final outcome of meet
15 February 2026	Online	Divided the tasks among ourselves.
11 March 2026	Offline	Progress on individual parts was discussed.
18 March 2026	Offline	Discussion on backend, Frontend and database review
24 March 2026	Online	Code review and work on backend features
26 March 2026	Offline	overall integration, bug correction and discussion on implementation document